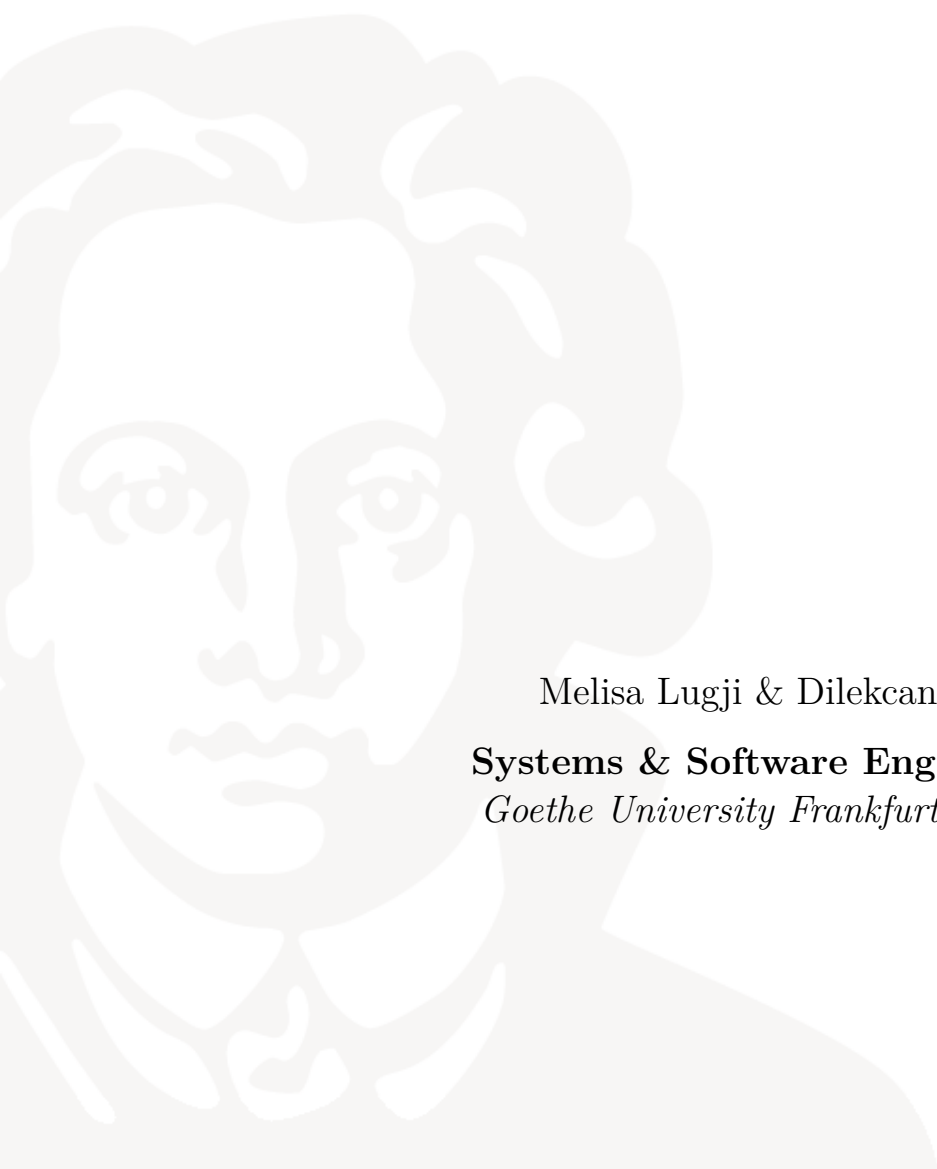


Project Report: DriftDetect



Melisa Lugji & Dilekcan Pamir

Systems & Software Engineering I

Goethe University Frankfurt am Main

Contents

1	Reader's Guide	1
2	One-page Overview	1
3	Users and Benefits	1
4	Operational Concept and Workflow	2
4.1	Production mode workflow	2
4.2	Engineering mode workflow	2
4.3	System context diagram	3
5	Scope and Context	3
5.1	In scope (MVP)	3
5.2	Out of scope (v1)	3
5.3	Dashboard interface	3
6	Artifacts and Interfaces	3
7	Terminology and Notation	5
8	Lifecycle Choice: V-Model and Verification Strategy	5
8.1	Why the V-Model fits	5
8.2	V-Model mapping for DriftDetect	5
8.3	Verification legend	6
9	System Design and Implementation Summary	6
9.1	Baseline profile	6
9.2	Drift detection approach	6
9.3	Statistical evidence	8
9.4	Severity model	8
9.5	Global decision logic	8
9.6	Default detector configuration	8
9.7	Simulation	9
9.8	Dashboard	9
10	Needs	9
11	Use Cases	9
12	Requirements	10
12.1	Functional requirements	10
12.2	Non-functional requirements	11
13	Coverage Report Definition	11
13.1	Motivation	11
13.2	Inputs and outputs	12
13.3	Core objects and ground truth	13
13.4	Severity and global drift decision	13
13.5	Coverage metrics	14
13.5.1	False Positive Rate (FPR)	14
13.5.2	Time-to-Detect (TTD)	14
13.5.3	Power within tolerance	14

13.5.4 Missed events	14
13.6 Interpretation	14
14 Evaluation Results	14
14.1 Income experiment	14
14.2 Monitoring blind spot	15
15 Acceptance Criteria	16
16 RTM (Needs → Requirements → Tests)	17
17 Risks	17
18 Conclusion	17

1 Reader's Guide

This document combines two purposes:

- it specifies DriftDetect as a software system, and
- it reports the implemented design and project results.

Recommended reading order.

- Sections 1–4 explain the system goal, users, workflows, and scope.
- Sections 5–8 define artifacts, terminology, lifecycle choice, and system design.
- Sections 9–11 list needs, use cases, and requirements.
- Section 12 defines coverage formally using ground-truth drift events.
- Sections 13–16 summarize implementation results, verification, risks, and conclusion.

2 One-page Overview

Goal. DriftDetect is a Python toolkit for tabular drift monitoring. It:

1. detects **data drift** and **schema drift** between a baseline and new data windows,
2. simulates controlled drift scenarios with ground truth (“drift as code”),
3. computes a **coverage report** that quantifies detector quality using false positive rate, detection power, time-to-detect, and missed events,
4. produces machine-readable artifacts and human-readable views.

Why this is different. Many drift tools focus only on reporting that drift happened. DriftDetect also supports **engineering-mode validation**: controlled scenarios can stress-test a monitoring setup and reveal its blind spots.

Operational modes.

- **Production mode:** baseline → detect drift on new windows.
- **Engineering mode:** simulate drift → baseline → detect drift → evaluate coverage.

Main artifacts. The system produces stable artifacts such as `baseline.json`, `drift.json`, `drift_*.json`, `manifest.json`, `drift_log.json`, `coverage.json`, and `report.html`. These files make the workflow reproducible, automatable, and easy to visualize in the dashboard.

3 Users and Benefits

Primary users.

- **ML Engineer:** wants early warning when incoming data changes so models do not silently degrade.
- **Data Engineer:** wants a reproducible way to validate pipelines and detect schema breaks.
- **CI / automation workflow:** wants machine-readable outputs and pass/fail signals.

Main benefits.

- Detect drift and schema changes with explicit evidence.
- Measure monitoring quality using controlled scenarios instead of intuition.
- Produce artifacts that are stable, testable, and easy to use in automation or visualization.

4 Operational Concept and Workflow

This section explains the intended end-to-end workflow in the order a user experiences it.

4.1 Production mode workflow

1. **Baseline build:** the user runs a CLI command on historical trusted data to produce `baseline.json`.
2. **Window detection:** for each new window dataset (CSV/Parquet), the detector produces `drift.json`.
3. **Optional visualization:** the Streamlit dashboard loads artifacts and displays results. It does not recompute drift metrics.

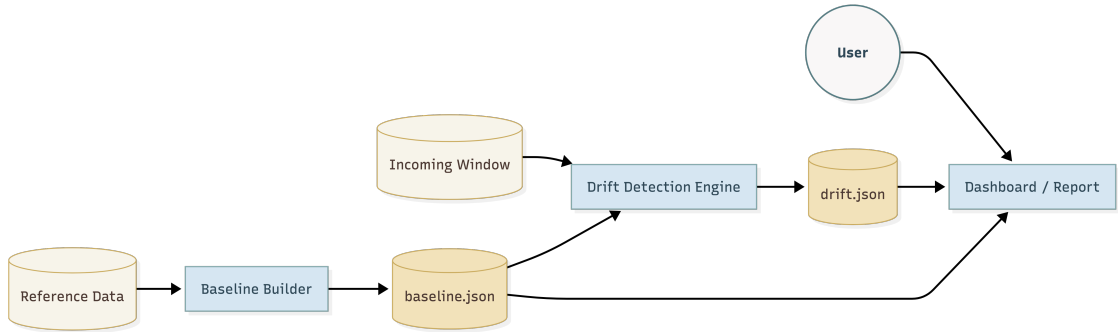


Figure 1: Production-mode workflow in DriftDetect.

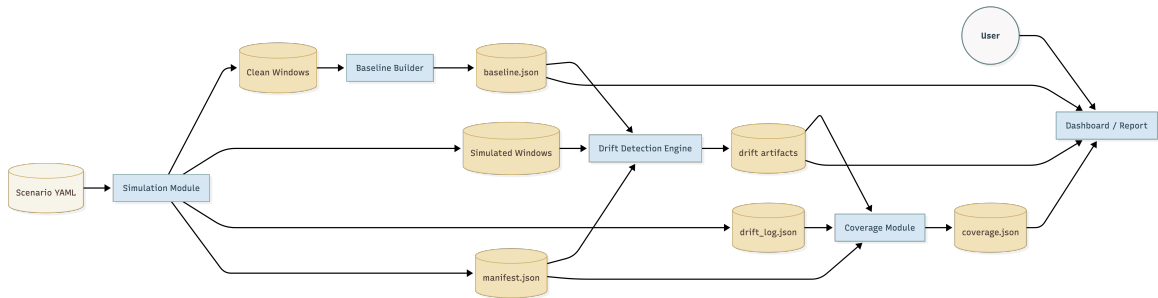


Figure 2: Engineering-mode workflow in DriftDetect.

4.2 Engineering mode workflow

1. **Scenario definition:** the user writes a YAML file describing drift events and intensities.
2. **Simulation:** the simulator generates windows and a ground-truth drift log.
3. **Detection replay:** the detector runs on each simulated window using the same logic as in production mode.
4. **Coverage computation:** the system compares detector outcomes to ground truth and writes `coverage.json`.

4.3 System context diagram

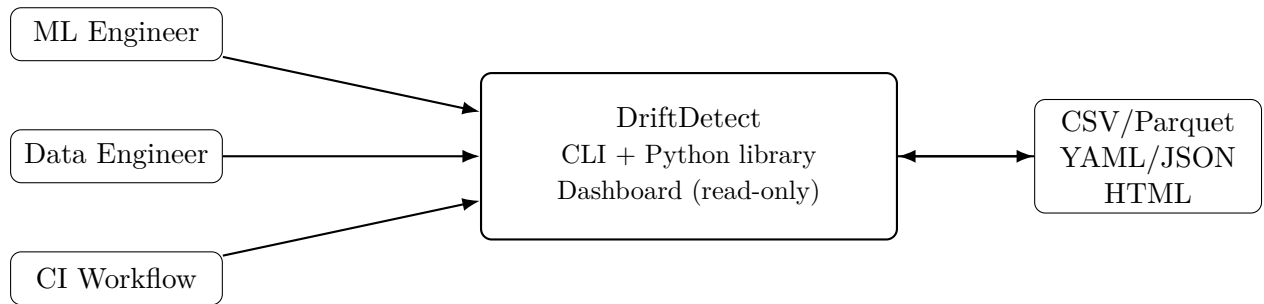


Figure 3: High-level context. Users interact with DriftDetect via CLI or Python; results are stored as artifacts and may be visualized in the dashboard.

5 Scope and Context

5.1 In scope (MVP)

- Baseline profiling (schema + per-feature summaries)
- Drift detection for numeric and categorical features
- Schema drift detection
- Drift simulation with scenario YAML and ground-truth drift log
- Coverage report (FPR, power, time-to-detect, missed events)
- CLI + JSON outputs + HTML report
- Streamlit dashboard for visualizing drift and coverage artifacts (reads JSON outputs; no recomputation)

5.2 Out of scope (v1)

- Concept drift requiring labels or model-performance monitoring
- Streaming and message-bus integration
- Multi-user monitoring platform features such as authentication and alert routing

5.3 Dashboard interface

The dashboard is a **visualization layer**. It loads artifacts generated by the CLI, such as `baseline.json`, `drift.json`, and `coverage.json`, and presents interactive views for humans. In the MVP, the dashboard does not execute baseline building, drift detection, simulation, or coverage computation.

6 Artifacts and Interfaces

To reduce ambiguity, this section defines the core artifact files as the system's stable interface.

Artifact	Produced by	Purpose and required contents (summary)
<code>baseline.json</code>	CLI baseline-build	Baseline profile: schema, per-feature summaries, missing rates, metadata.
<code>drift.json</code>	CLI detect	Window result: schema drift, per-feature metrics and severity, feature ranking, global drift decision.
<code>drift_*.json</code>	CLI detect-suite	Drift results for multiple windows in engineering mode.
<code>scenario.yaml</code>	User input	Scenario definition: seed, window settings, drift events, onset windows, and intensity.
<code>manifest.json</code>	CLI simulate	Generated window manifest with dataset, paths, seed, and scenario metadata.
<code>drift_log.json</code>	CLI simulate	Ground truth: event ids, types, onset windows, affected features, intensity.
<code>coverage.json</code>	CLI coverage	Coverage metrics: suite-level FPR, power, TTD stats, missed events list, per-event results.
<code>report.html</code>	CLI report	Human-readable view of drift and coverage results.

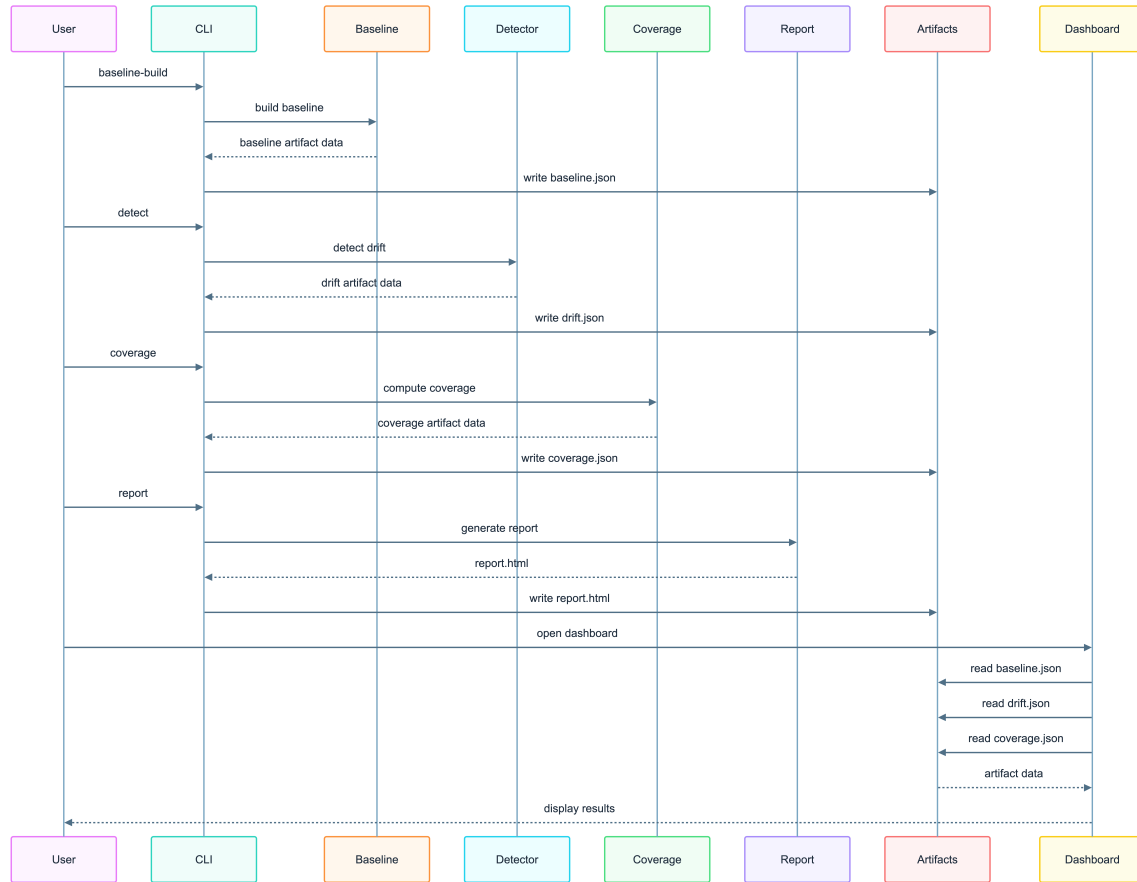


Figure 4: Runtime interaction view of DriftDetect.

7 Terminology and Notation

To avoid ambiguity, the following terms are used consistently:

- **Data drift**: distribution change of input features between baseline and a window.
- **Schema drift**: dataset structure change (added/removed columns or type changes).
- **Dataset window** w_i : batch of records for time/batch index i .
- **Baseline profile** B : stored reference summary built from baseline data.
- **Scenario** q : specification that generates ordered windows $W_q = \langle w_0, \dots, w_{N-1} \rangle$ plus ground truth.
- **Scenario suite** $\mathcal{Q}$: set of scenarios used for coverage evaluation.
- **Drift event** e : ground-truth record with `event_id`, type $t(e)$, onset $s(e)$, affected features, and intensity.
- **Ground-truth drift log**: artifact containing all drift events for a scenario.
- **Tolerance** M : maximum allowed detection delay (in windows) to count an event as detected.
- **Global drift flag** $g(w_i) \in \{0, 1\}$: binary decision for window w_i .
- **Coverage**: monitoring coverage, not code coverage.
- **CLI**: command-line interface for running DriftDetect.
- **CI**: continuous integration; automated checks run on each change.

8 Lifecycle Choice: V-Model and Verification Strategy

8.1 Why the V-Model fits

- Drift detection is correctness-sensitive: outputs must be trustworthy and reproducible.
- The simulator provides ground truth drift events, enabling measurable verification.
- The V-model aligns naturally with traceability from requirements to tests.

8.2 V-Model mapping for DriftDetect

Specification side	Verification side
Needs	Validation: demo + stakeholder checklist
System requirements	Acceptance tests: CLI + artifacts
Architecture and interfaces	Integration tests: simulate→detect→coverage→report
Detailed design and algorithms	Unit tests: metrics, transforms, serialization

8.3 Verification legend

The **Verify** column in the requirements section describes how compliance is checked:

- **Fit criterion:** measurable expected outcome.
- **Method codes:**
 - **Unit T:** unit test of a function or module
 - **Int T:** integration test across multiple modules
 - **Acc T:** acceptance test via CLI producing artifacts
 - **I:** inspection
 - **D:** demonstration

9 System Design and Implementation Summary

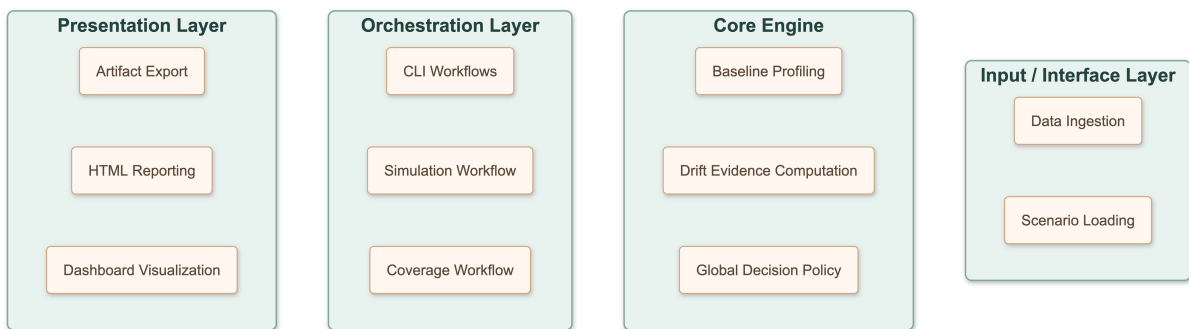


Figure 5: Layered architecture of DriftDetect.

9.1 Baseline profile

The baseline profile is built from clean reference data and stored as `baseline.json`. It captures the normal state of the dataset and contains:

- schema information,
- numeric summaries such as mean, standard deviation, and quantiles,
- numeric histograms,
- categorical frequencies,
- missingness rates.

9.2 Drift detection approach

Detection is performed at two levels:

- **Schema level** – added, removed, or type-changed features
- **Feature level** – numeric and categorical drift relative to the baseline

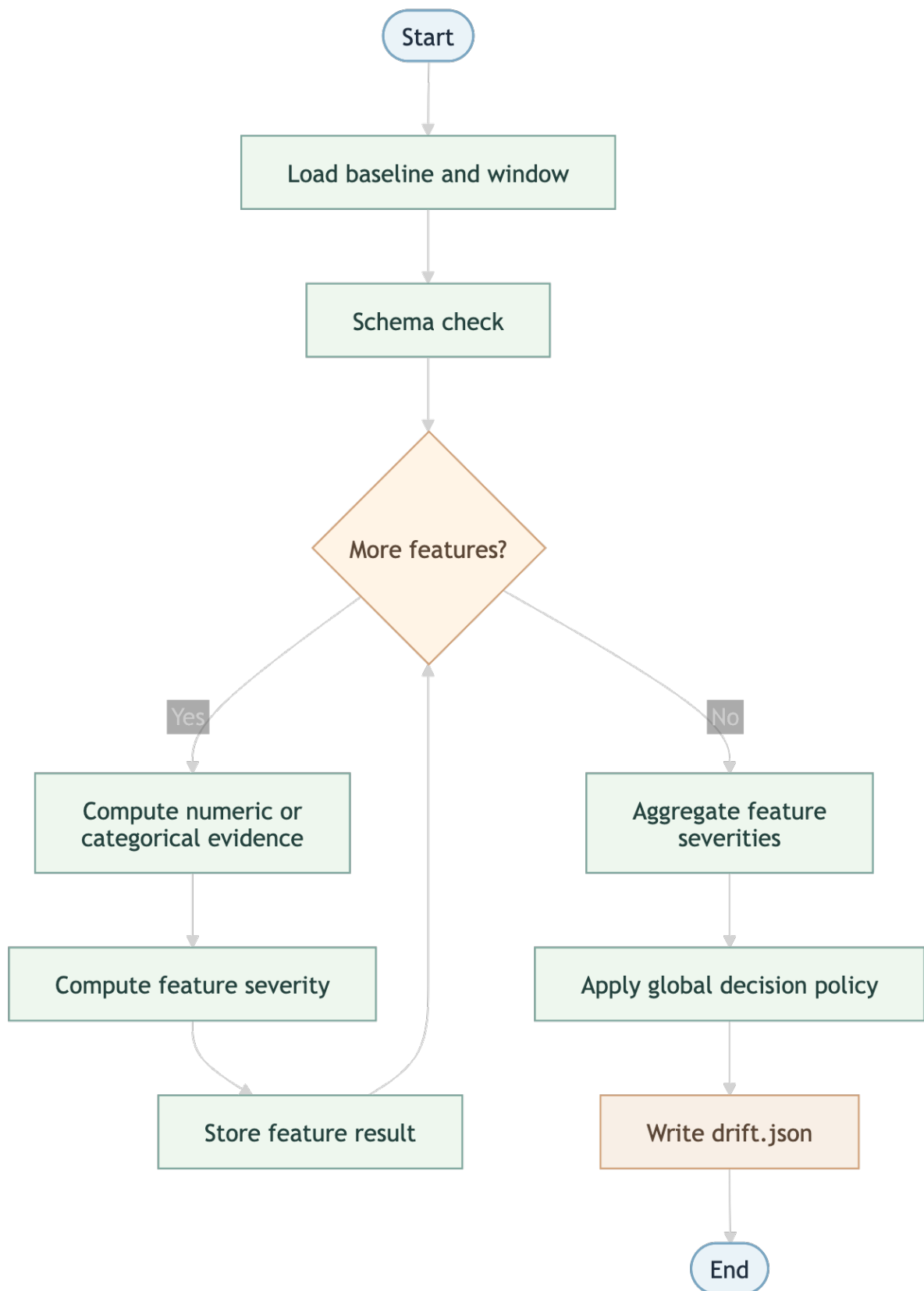


Figure 6: High-level detection workflow in DriftDetect, from baseline comparison to the final `drift.json` artifact.

9.3 Statistical evidence

Numeric features. The implementation uses:

- Kolmogorov–Smirnov distance,
- Wasserstein distance,
- mean shift,
- standard deviation change,
- missingness change.

Categorical features. The implementation uses:

- Jensen–Shannon divergence,
- chi-square p-value mapping,
- missingness change.

9.4 Severity model

Each feature receives a severity score in $[0, 1]$.

Numeric feature severity.

$$\text{sev}_{\text{numeric}}(f) = \max(s_{KS}, s_{WD}, s_{\text{mean}}, s_{\text{std}}, s_{\text{miss}})$$

Categorical feature severity.

$$\text{sev}_{\text{categorical}}(f) = \max(s_{JS}, s_p, s_{\text{miss}})$$

A feature is marked as drifted if:

$$\text{sev}(f) \geq S_{\text{feat}}$$

9.5 Global decision logic

The final window-level decision uses:

- S_{feat} – feature severity threshold
- S_{global} – global severity threshold
- K – minimum number of drifted features

A window is flagged if:

$$\text{schema drift present} \quad \vee \quad \max_f \text{sev}(f) \geq S_{\text{global}} \quad \vee \quad |\{f : \text{sev}(f) \geq S_{\text{feat}}\}| \geq K$$

9.6 Default detector configuration

The current detector configuration includes:

$$\alpha = 0.05, \quad S_{\text{feat}} = 0.7, \quad S_{\text{global}} = 0.7, \quad K = 3$$

and the normalization parameters

$$\tau_{WD} = 1.0, \quad \tau_{\mu} = 1.0, \quad \tau_{\sigma} = 0.5, \quad \tau_{\text{miss}} = 0.10$$

9.7 Simulation

Engineering mode uses controlled drift scenarios defined in YAML. The simulator supports the drift modes used in the project examples, including abrupt and gradual behavior.

The implemented example scenarios generate:

- windows/window_000.csv ... window_011.csv
- manifest.json
- drift_log.json

The project examples use two dataset themes:

- **income** – synthetic demographic data inspired by the UCI Adult Income dataset structure
- **logistics** – synthetic order-pattern data generated using the course order simulator

9.8 Dashboard

The dashboard is an artifact-only visualization layer. It reads JSON artifacts and never recomputes drift logic. It supports:

- experiment selection,
- severity-over-time views,
- distribution comparison,
- ranked feature tables,
- coverage summaries.

10 Needs

Needs express *why* the system exists; requirements express *what* it must do.

Need ID	Need statement
NEED-001	Detect when incoming data deviates meaningfully from a trusted baseline.
NEED-002	Produce CI-friendly outputs and pass/fail signals (machine readable).
NEED-003	Provide reproducible drift simulation with ground truth drift events.
NEED-004	Provide traceability: needs → requirements → tests.
NEED-005	Provide explainable evidence of what changed (not only a score).
NEED-006	Demonstrate software engineering practices (spec, tests, CI, docs).

11 Use Cases

UC	Description
UC-01	Build baseline profile from dataset.
UC-02	Detect drift on a new window vs baseline.
UC-03	Simulate drift scenarios from YAML (generate windows + drift log).
UC-04	Run coverage report over a scenario suite (FPR, power, TTD, missed events).
UC-05	Render HTML report from JSON results.

UC-06	CI gate: pass/fail based on drift policy or coverage thresholds.
UC-07	View drift and coverage results in the Streamlit dashboard (artifact-based visualization).

12 Requirements

Requirements are stated as **shall** statements, each paired with a fit criterion and a verification method. Coverage definitions are in Section 13, and the severity model/global policy defining $g(w_i)$ is in Section 13.4.

12.1 Functional requirements

ID	Prio	Requirement	Verify (fit + method)
REQ-001	MUST	DriftDetect shall read datasets from CSV and Parquet files.	Fit: loads fixtures. Acc T
REQ-002	MUST	DriftDetect shall build a baseline profile with schema, per-feature summaries, histograms/frequencies, and missing rates.	Fit: required JSON keys present. Acc T
REQ-003	MUST	DriftDetect shall detect schema drift (added/removed/type-changed columns).	Fit: matches fixture truth. Acc T
REQ-004	MUST	DriftDetect shall compute numeric drift evidence (KS, Wasserstein, mean shift, std change, missingness).	Fit: values within tolerance. Unit T
REQ-005	MUST	DriftDetect shall compute categorical drift evidence (JS divergence, chi-square p mapping, missingness).	Fit: values within tolerance. Unit T
REQ-006	MUST	DriftDetect shall compute per-feature severity in $[0, 1]$ and rank features by severity.	Fit: bounds + correct ordering. Unit T
REQ-007	MUST	DriftDetect shall support configurable thresholds and a global drift policy for CI gates.	Fit: expected global flag for fixtures. Acc T
REQ-008	MUST	DriftDetect shall load scenario YAML defining windows, seed, drift events, and onset.	Fit: schema validation passes/fails correctly. Unit T
REQ-009	MUST	DriftDetect shall simulate the implemented drift scenario types used in project examples, including numeric shift, variance shift, missingness, and new category drift.	Fit: measured shift matches config. Unit T
REQ-010	MUST	DriftDetect shall output a ground-truth drift log with <code>event_id</code> , <code>type</code> , <code>onset</code> , <code>features</code> , <code>intensity</code> .	Fit: matches scenario YAML. Acc T
REQ-011	MUST	DriftDetect shall run detection across simulated windows and record per-window outcomes.	Fit: N windows $\rightarrow$ N results. Int T
REQ-012	MUST	DriftDetect shall compute coverage metrics: FPR, power, and time-to-detect.	Fit: matches fixture suite. Int T

ID	Prio	Requirement	Verify (fit + method)
REQ-013	MUST	DriftDetect shall list missed events not detected within tolerance M .	Fit: correct missed list. Int T
REQ-014	MUST	DriftDetect shall output JSON artifacts and render an HTML report with top-feature evidence.	Fit: required sections exist. Acc D
REQ-015	MUST	DriftDetect shall provide CLI commands <code>baseline/detect/simulate/coverage/report</code> with <code>-help</code> .	Fit: help works + commands run. Acc T
REQ-016	MUST	DriftDetect shall provide a reproducible end-to-end example workflow.	Fit: demo runs. D
REQ-017	SHOULD	DriftDetect shall provide a Streamlit dashboard that loads <code>baseline.json</code> , <code>drift.json</code> , and <code>coverage.json</code> from local files.	Fit: loads without errors. Acc D
REQ-018	SHOULD	Dashboard shall provide a Monitoring view: global flag, schema drift summary, ranked features, drill-down.	Fit: widgets visible for fixture. Acc D
REQ-019	SHOULD	Dashboard shall provide a Coverage view: suite FPR, power, TTD summary, missed events, drill-down.	Fit: widgets visible for fixture. Acc D
REQ-020	MUST	Dashboard shall not recompute drift metrics; it shall only visualize artifacts.	Fit: no compute actions. I

12.2 Non-functional requirements

ID	Prio	Requirement
NFR-001	MUST	Reproducibility: same inputs + seed shall produce identical simulation outputs.
NFR-002	MUST	Traceability: each MUST requirement shall map to at least one test in RTM.
NFR-003	MUST	CI gate: tests and checks shall pass before merge.
NFR-004	MUST	Usability: CLI errors shall be actionable.
NFR-005	SHOULD	Maintainability: modular code structure shall separate detection, simulation, evaluation, and visualization.
NFR-006	SHOULD	Explainability: feature-level metrics and summaries shall remain available.
NFR-007	SHOULD	Portability: file-based artifacts shall be usable across environments.
NFR-008	SHOULD	Dashboard performance should remain acceptable for typical project-scale experiments.

13 Coverage Report Definition

Coverage refers to *monitoring coverage*: how well a detector configuration identifies relevant drift while avoiding false alarms.

13.1 Motivation

Given a baseline distribution P_0 estimated from historical data, the system inspects each window w_i and decides whether it is out-of-distribution relative to P_0 . Any detector must balance:

- false positives: alert when nothing changed,
- misses: fail to alert when drift exists.

Coverage quantifies this trade-off using controlled scenarios with ground truth.

13.2 Inputs and outputs

Inputs:

- baseline profile B ,
- detector configuration C ,
- scenario suite Q with windows and ground truth.

Outputs:

- `coverage.json`

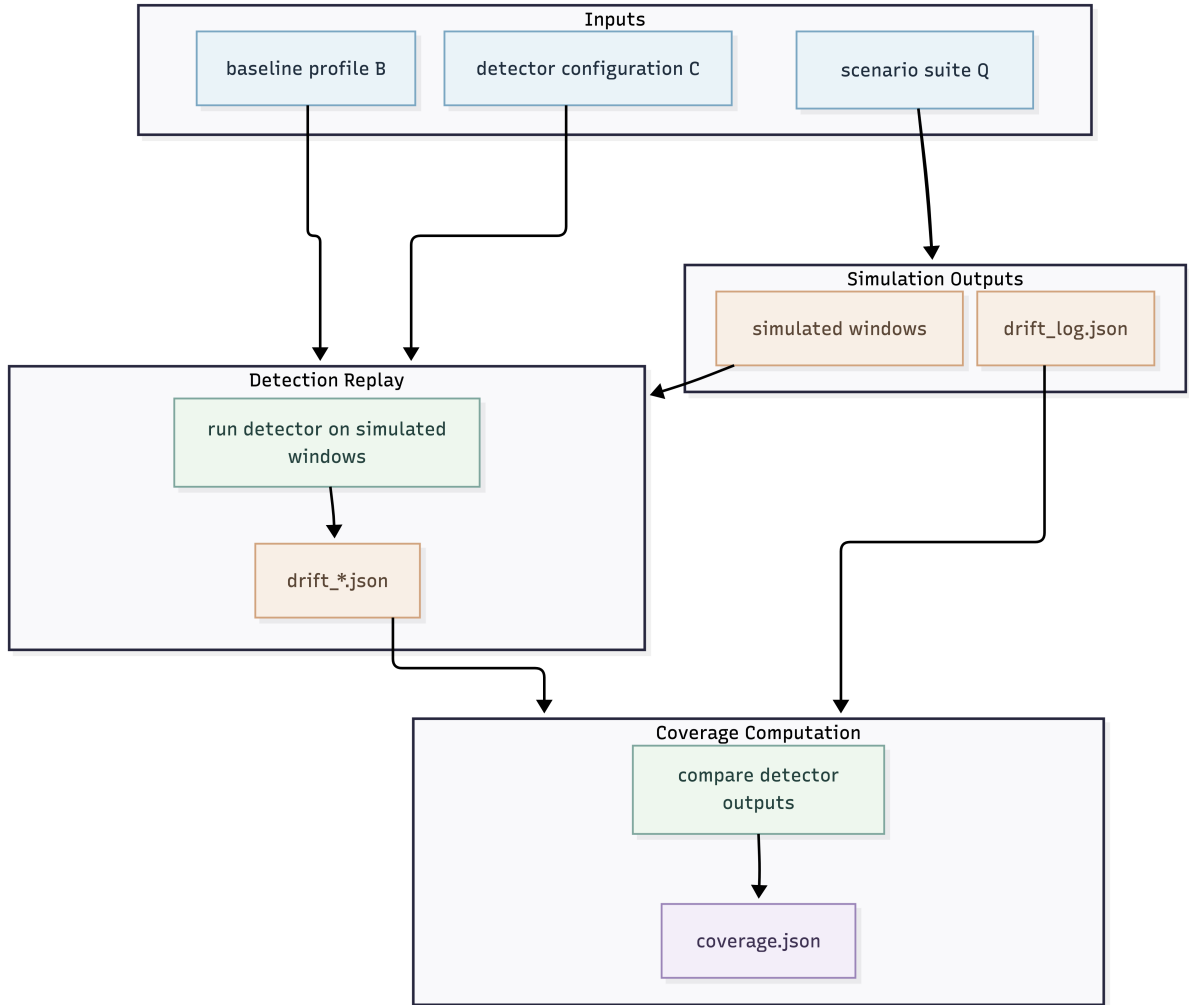


Figure 7: Evaluation pipeline in DriftDetect.

13.3 Core objects and ground truth

Windows. A scenario q consists of N ordered windows:

$$W_q = \langle w_0, w_1, \dots, w_{N-1} \rangle.$$

Ground-truth drift events. Each scenario q has a drift log:

$$E_q = \{e_1, e_2, \dots, e_K\}.$$

Each event contains `event_id`, `type`, `onset`, `features`, and `intensity`.

Active drift windows.

$$\text{active}(e, i) \iff i \geq s(e), \quad \text{drift}(q, i) \iff \exists e \in E_q : \text{active}(e, i).$$

Tolerance M . An event counts as detected if drift is flagged within M windows after onset.

13.4 Severity and global drift decision

Coverage metrics use the global drift flag $g(w_i) \in \{0, 1\}$.

Metric-to-severity normalization. Numeric:

$$s_{WD} = \text{clip}\left(\frac{WD^*}{\tau_{WD}}, 0, 1\right), \quad s_{mean} = \text{clip}\left(\frac{z_\mu}{\tau_\mu}, 0, 1\right), \quad s_{std} = \text{clip}\left(\frac{r_\sigma}{\tau_\sigma}, 0, 1\right)$$

with missingness component

$$s_{miss} = \text{clip}\left(\frac{\Delta_{miss}}{\tau_{miss}}, 0, 1\right)$$

and categorical p-value mapping

$$s_p = \text{clip}\left(\frac{\alpha - p}{\alpha}, 0, 1\right).$$

Per-feature severity. For numeric features:

$$\text{sev}_{\text{numeric}}(f) = \max(s_{KS}, s_{WD}, s_{mean}, s_{std}, s_{miss})$$

For categorical features:

$$\text{sev}_{\text{categorical}}(f) = \max(s_{JS}, s_p, s_{miss})$$

Feature drift flag.

$$\text{drifted}_i(f) \iff \text{sev}_i(f) \geq S_{\text{feat}}$$

Global drift policy. Let $SD(w_i)$ indicate schema drift. Then:

$$g(w_i) = 1 \iff \left(SD(w_i) = 1 \right) \vee \left(\max_f \text{sev}_i(f) \geq S_{\text{global}} \right) \vee \left(|\{f : \text{drifted}_i(f)\}| \geq K \right)$$

Default configuration.

$$\alpha = 0.05, \quad S_{\text{feat}} = 0.7, \quad S_{\text{global}} = 0.7, \quad K = 3$$

$$\tau_{WD} = 1.0, \quad \tau_\mu = 1.0, \quad \tau_\sigma = 0.5, \quad \tau_{miss} = 0.10$$

13.5 Coverage metrics

13.5.1 False Positive Rate (FPR)

$$\mathcal{I}_q^{(0)} = \{i \mid \neg \text{drift}(q, i)\}, \quad \text{FPR}(q) = \frac{\sum_{i \in \mathcal{I}_q^{(0)}} \mathbf{1}[g(w_i) = 1]}{|\mathcal{I}_q^{(0)}|}$$

13.5.2 Time-to-Detect (TTD)

$$d(e) = \min\{i \mid i \geq s(e) \wedge g(w_i) = 1\}, \quad \text{TTD}(e) = d(e) - s(e)$$

If no detection exists, then $\text{TTD}(e) = \infty$.

13.5.3 Power within tolerance

$$\text{detected}_M(e) \iff \text{TTD}(e) \leq M, \quad \text{Power}_M(\mathcal{Q}) = \frac{\sum_{q \in \mathcal{Q}} \sum_{e \in E_q} \mathbf{1}[\text{detected}_M(e)]}{\sum_{q \in \mathcal{Q}} |E_q|}$$

13.5.4 Missed events

An event is missed if it is not detected within M windows. The report lists event id, type, onset, affected features, and intensity.

13.6 Interpretation

Coverage answers: *Given configuration C , how often do we false alarm, how often do we catch true drift, and how fast do we detect it?*

14 Evaluation Results

14.1 Income experiment

The income experiment used a synthetic demographic dataset inspired by the UCI Adult Income dataset structure (features: age, income, hours_per_week, education_num, occupation, marital_status). The baseline was built from 2000 independently generated rows; the scenario ran 12 windows of 600 rows each with 4 clean reference windows.

Event	Drift Type	Power	TTD	Primary test
Income gradual shift	numeric_shift	1.0	2 windows	KS + Wasserstein
New occupation category	new_category	1.0	0 windows	JSD = 1.0
Hours variance drift	variance_shift	1.0	0 windows	Wasserstein

Overall FPR = 0.0 and Power = 1.0. The gradual income shift required TTD = 2 windows; abrupt events were detected immediately. Figure 8 shows the dashboard alert and Figure 9 the feature severity ranking.



Figure 8: Dashboard alert and metric summary. Max severity = 1.0, drifted features = 2/6, schema unchanged.

All Features							
	Feature	Type	Severity	Drifted	KS	WD	JS
0	hours_per_week	numeric	1.000000	✓	0.195000	6.427000	None
1	occupation	categorical	1.000000	✓	None	None	0.146000
2	income	numeric	0.646111	—	0.437000	18238.979000	None
3	education_num	numeric	0.094126	—	0.030000	0.138000	None
4	age	numeric	0.053361	—	0.029000	0.631000	None
5	marital_status	categorical	0.001358	—	None	None	0.001000

Figure 9: Feature severity ranking. `hours_per_week` and `occupation` at severity 1.0; `income` at 0.646; remaining features near zero.

Figure 10 shows the occupation distribution. `Gig_Economy` has no baseline bar – absent from training data, producing $JSD = 1.0$ immediately. Figure 11 shows the income distribution shift.

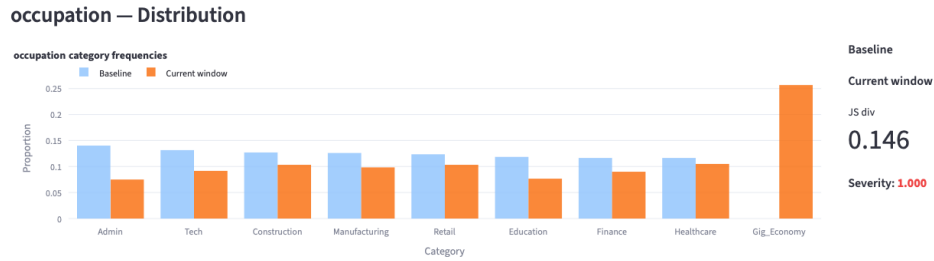


Figure 10: Occupation distribution comparison. `Gig_Economy` has no baseline representation; $JSD = 0.146$, severity = 1.0.

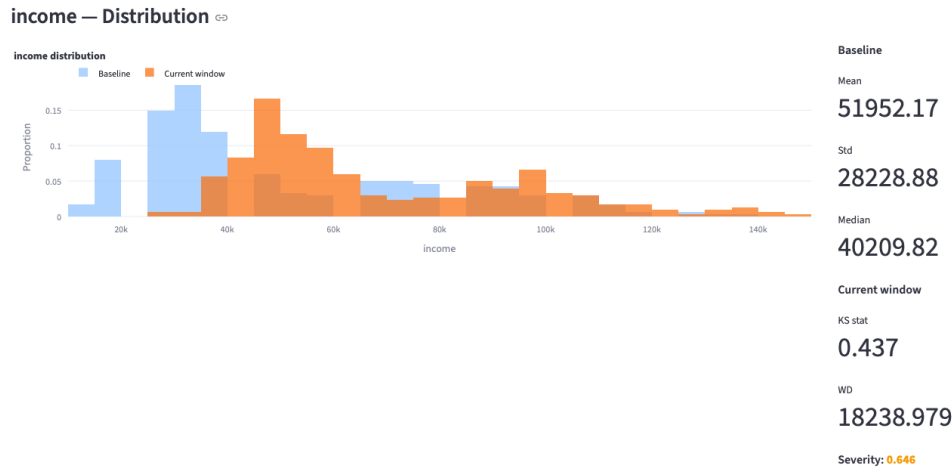


Figure 11: Income distribution comparison. Blue = baseline (mean \$51,952), orange = drifted window. $KS = 0.437$, Wasserstein = 18,238, severity = 0.646.

14.2 Monitoring blind spot

The logistics experiment used the order simulator with two customer profiles. Three scenarios were tested: weight mean shift (`CUST_A`), new unit type (`CUST_B`), and day-pattern shift (`CUST_A` Tuesday $\rightarrow$ Friday). The first two were detected immediately with Power = 1.0. The day-pattern shift was completely missed:

- $\text{FPR} = 0.0$, $\text{Power} = 0.0$, $\text{TTD} = \infty$, one missed event

Figure 12 shows the coverage report. The root cause is signal dilution: CUST_B orders uniformly across all seven days, masking CUST_A's shift in the pooled window. Maximum Day severity reached 0.456 – below threshold. The fix: monitor per-customer sub-windows instead of pooled data.

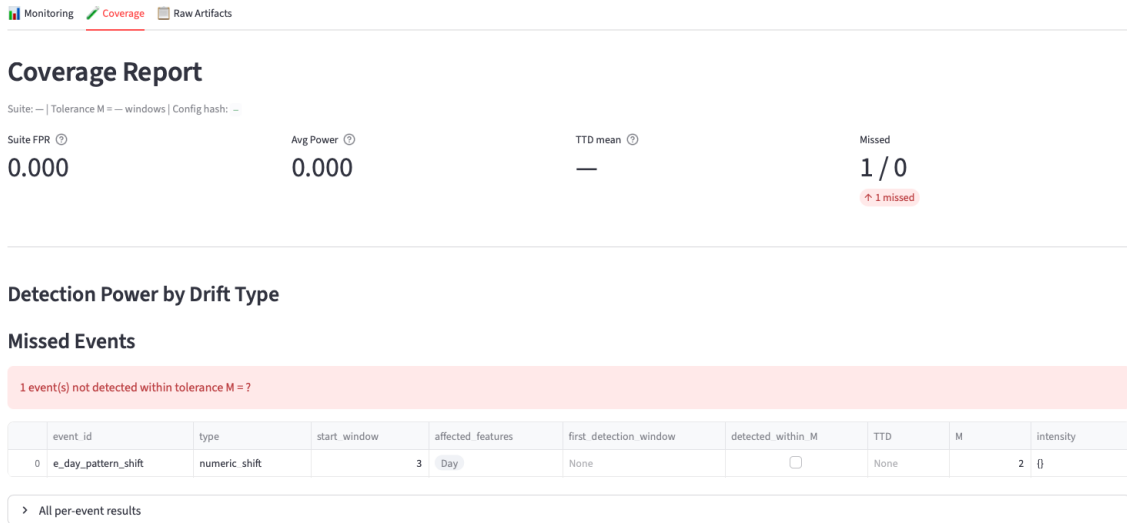


Figure 12: Coverage report for the logistics day-pattern shift. $\text{FPR} = 0.0$, $\text{Power} = 0.0$, $\text{TTD} = \infty$, `first_detection_window`: None.

This demonstrates the core value of engineering mode: simulation and coverage can reveal blind spots that production-only monitoring cannot detect.

15 Acceptance Criteria

- AC-01: Baseline save/load preserves required fields.
- AC-02: Scenario onset at window 10 activates drift at window 10.
- AC-03: A no-drift suite yields FPR below the configured threshold.
- AC-04: A controlled suite yields expected Power and TTD values.
- AC-05: HTML report includes metadata, schema drift, and top features with baseline vs. current summaries.
- AC-06: Dashboard loads `drift.json` and `coverage.json` and displays required widgets.
- AC-07: Dashboard remains artifact-only and performs no drift computation.

16 RTM (Needs → Requirements → Tests)

Need	Req	Use Case	Test IDs	Test level (V)
001	REQ-004	UC-02	TC-DE-010/011	Unit
001	REQ-005	UC-02	TC-DE-020/021	Unit
002	REQ-007	UC-06	TC-DE-040	Acceptance
003	REQ-009	UC-03	TC-SIM-020..024	Unit
003	REQ-012	UC-04	TC-COV-010/020/030	Integration
005	REQ-014	UC-05	TC-REP-020/021	Acceptance
006	NFR-003	All	TC-CI-001	Acceptance
005	REQ-018	UC-07	TC-DASH-010	Acceptance
003	REQ-019	UC-07	TC-DASH-020	Acceptance
002	REQ-020	UC-07	TC-DASH-001	Inspection

17 Risks

Risk	Mitigation
Gradual drift is detected late	Evaluate configurations with Time-to-Detect (TTD) and compare thresholds using controlled scenarios.
Monitoring blind spots in aggregated data	Use simulation and coverage analysis to reveal missed events; refine monitoring granularity (e.g. per-segment or per-customer views).
Threshold sensitivity	Calibrate S_{feat} , S_{global} , and K through coverage evaluation instead of choosing them ad hoc.
Reproducibility bugs	Use fixed seeds, stable artifacts, and artifact-based verification across runs.
Scope creep	Keep a strict MVP: tabular data, artifact-based workflow, CLI execution, and read-only dashboard.

18 Conclusion

DriftDetect is a complete drift monitoring workflow for tabular data. It combines baseline profiling, schema and feature drift detection, simulation, coverage evaluation, and visualization in one artifact-driven pipeline.

The project shows that drift monitoring should go beyond detecting that something changed. With controlled simulation and ground truth, detector quality becomes measurable through FPR, detection power, TTD, and missed events.

A key project finding was that the logistics day-pattern shift was completely missed when data was monitored only in aggregated form. Coverage analysis exposed this blind spot and pointed directly to a better design choice: more fine-grained monitoring.

Overall, DriftDetect provides a reproducible workflow for validating and improving drift monitoring setups.